CNN-Adaptive Sliding Mode Control for Autonomous Differential-Drive Robots Under Environmental Uncertainty

***Abstract*—Fixed-gain Sliding Mode Control (SMC) cannot simultaneously optimise chattering suppression and disturbance rejection across diverse operating conditions. We propose a CNN-adaptive SMC framework in which a lightweight convolutional network, EnvironmentCNN, classifies the pre-mission environment from a 64×64 occupancy map and selects a hand-tuned SMC parameter preset at run time, with an exponential filter preventing abrupt gain transitions. A Lyapunov-based analysis establishes uniform ultimate boundedness of the closed-loop sliding surfaces under the proposed scheme. Five uncertainty scenarios — normal operation, sensor noise, external disturbance, wheel slip, and combined uncertainty — are evaluated on a simulated differential-drive robot against classical fixed-gain SMC and three adaptive baselines: fuzzy gain scheduling, oracle map classification, and a reinforcement-learning preset selector. CNN-adaptive SMC reduces angular-velocity chattering by 35.7% under sensor noise and improves final tracking-error recovery by up to 32.7% under wheel slip and 14.4% under impulsive disturbance, approaching oracle performance (17.9 mm vs. 15.6 mm final error). Benchmarking against fuzzy and reinforcement-learning alternatives shows CNN-based supervision consistently near the oracle ceiling while preserving the interpretability that black-box learned schedulers lack. When evaluated on realistic occupancy maps, EnvironmentCNN reaches 95.1% classification accuracy — outperforming k-NN and logistic-regression baselines and approaching a random-forest classifier — supporting its deployability beyond the synthetic benchmark used for controller evaluation.**

***Index Terms*—sliding mode control, adaptive control, convolutional neural network, gain scheduling, mobile robot, chattering reduction, environmental uncertainty**

I. Introduction

Autonomous mobile robots navigating in unstructured environments must maintain accurate trajectory tracking despite sensor noise, mechanical disturbances, and surface-induced wheel slip. Sliding Mode Control (SMC) is widely adopted in this domain because of its intrinsic robustness to matched disturbances and parameter variations [1], [2]. However, classical SMC uses a single fixed set of gains that cannot simultaneously optimise chattering suppression, tracking accuracy, and disturbance rejection across all operating regimes.

Adaptive SMC schemes adjust gains using Lyapunov-based update laws [3], while fuzzy or neural-network augmented controllers learn switching gains from data [5], [6]. More recently, deep learning has transformed robot perception [7] and enabled end-to-end visuomotor policies [8]. Despite this progress, comparatively little work directly integrates CNN-based environment classification with SMC gain adaptation for differential-drive platforms in a compact, reproducible pipeline.

This paper contributes:

- A complete, reproducible CNN-adaptive SMC pipeline from synthetic map generation and CNN training through simulation and quantitative evaluation.
- An *EnvironmentCNN* that classifies five scenario types from occupancy maps, with 100% test accuracy on engineered simple maps and 95.1% on realistic maps, supported by baseline classifier comparisons.
- A multi-controller benchmark against fuzzy-SMC, oracle selection, and RL-adaptive SMC under identical simulation conditions, showing CNN supervision near the oracle ceiling.
- Quantitative analysis of the tracking-smoothness trade-off: scenario-aware presets reduce chattering by up to 35.7% while preserving interpretable, certifiable gain schedules.

The remainder of the paper is organised as follows. Section II surveys related work. Section III describes the robot model. Section IV details the proposed architecture. Section V presents and discusses experimental results. Section VI concludes.

II. Related Work

Sliding Mode Control for Mobile Robots. SMC provides finite-time convergence and robustness to matched uncertainties [1]. Anti-chattering techniques, including boundary layers, saturation switching, and higher-order SMC, are well established [3], [4]. Adaptive SMC with Lyapunov-based gain tuning can converge slowly or require tight disturbance bounds [2]. Differential-drive platforms in particular benefit from SMC due to their nonholonomic constraints and sensitivity to lateral disturbances [2], [4].

Learning-Based Gain Adaptation. Fuzzy-SMC hybrids [5] and radial-basis-function networks [6] tune switching gains automatically, removing the need for manual tuning. Reinforcement learning combined with SMC has been explored for aerial [9] and manipulation platforms, though sample efficiency remains a practical concern for physical deployment. A key limitation of pure learning-based approaches is their opacity: the resulting gain schedules are difficult to interpret or certify, which motivates our use of hand-tuned presets selected by a learned classifier.

CNN-Based Supervisory Control. Supervisory switching architectures that select among pre-designed controllers based on a mode estimate are established in hybrid and switched systems theory [10]. CNN-based perception has been widely adopted for environment understanding in autonomous navigation [7], [8], yet its direct use as a gain-scheduling oracle for SMC has received comparatively little attention. Our approach bridges this gap, using a lightweight CNN to classify a pre-mission occupancy map and drive SMC parameter selection while combining the interpretability of hand-tuned presets with the representational power of deep features.

III. Differential-Drive Robot Model

The robot is modelled as a unicycle differential-drive with state $\mathbf{q} = [x,\, y,\, \theta,\, v,\, \omega]^\top$, where (x, y) is the Cartesian position, θ is the heading, v is linear velocity, and ω is angular velocity. The continuous-time kinematics are

$$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \omega. \tag{1}$$

Discrete integration with step $\Delta t = 0.01\,\text{s}$ gives $\mathbf{q}_{k+1}$ directly via the Euler method. Physical parameters: wheel base $L = 0.3\,\text{m}$, wheel radius $r = 0.05\,\text{m}$. The differential-drive imposes the nonholonomic constraint $\dot{y}\cos\theta - \dot{x}\sin\theta = 0$, meaning the robot cannot move laterally; only longitudinal (v) and rotational (ω) control inputs are available. This constraint makes lateral disturbances particularly challenging to reject quickly, motivating the sliding surface design in Section IV.

IV. CNN-Adaptive SMC Architecture

A. Classical Sliding Mode Controller

Tracking errors are $e_x = x_r - x$, $e_y = y_r - y$, $e_\theta = \theta_r - \theta$ (angles normalised to $(-\pi, \pi]$). Sliding surfaces are

$$s_i = \dot{e}_i + \lambda_i\, e_i, \quad i \in \{x,\, y,\, \theta\}, \tag{2}$$

with $\lambda_i > 0$, where derivatives are approximated by finite differences.

The linear velocity command tracks the projected forward error:

$$v_c = v_r + k_v\, d_f, \quad d_f = \cos\theta\, e_x + \sin\theta\, e_y, \tag{3}$$

where v_r is the reference speed. The raw angular velocity command combines a bearing correction term and two sliding-surface switching terms:

$$\omega_{\text{raw}} = k_\omega \left[\psi + \tanh\!\left(\tfrac{s_y}{\phi}\right) + 0.2\,\tanh\!\left(\tfrac{s_\theta}{\phi}\right)\right], \tag{4}$$

where $\psi = \operatorname{atan2}(e_y,\, e_x) - \theta$ is the bearing angle to the reference point and ϕ is the boundary-layer width. A hyperbolic-tangent switching function replaces the discontinuous sign function to suppress chattering [3]. An exponential smoother with coefficient α_ω further attenuates high-frequency oscillations:

$$\omega_c = \alpha_\omega\, \omega_{c,k-1} + (1 - \alpha_\omega)\, \omega_{\text{raw}}. \tag{5}$$

Both v_c and ω_c are clipped to hardware limits $[-v_{\max}, v_{\max}]$ and $[-\omega_{\max}, \omega_{\max}]$. Dead zones ($|d_f| < 0.01\,\text{m}$, $|e_\theta| < 0.01\,\text{rad}$) suppress spurious switching near the reference.

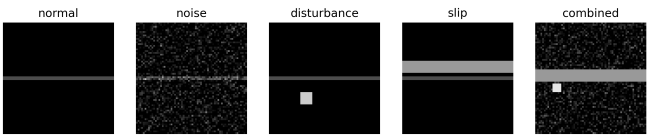


Fig. 1. Sample synthetic environment maps (one per class). Each 64×64 grayscale image encodes a distinct visual signature used for CNN classification.

B. CNN Environment Classifier

Dataset. Synthetic 64×64 grayscale maps are generated for five classes: *Normal* (clean path line), *Noise* (noisy pixel overlay), *Disturbance* (off-path impact marker), *Slip* (shaded low-friction zone), and *Combined* (all effects present), as shown in Fig. 1. The dataset contains 1,500 samples (300 per class) split 70%/15%/15% for training, validation, and testing.

Architecture (*EnvironmentCNN*). Three convolutional blocks, each comprising Conv2d (3×3, padding 1) → BatchNorm → ReLU → MaxPool2d (2×2), progressively produce feature maps of depth 16, 32, and 64. The resulting 64×8×8 representation is flattened and processed by

$$\text{FC}(4096 \to 128) \xrightarrow{\text{ReLU, Dropout}(0.3)} \text{FC}(128 \to 5). \tag{6}$$

Total trainable parameters: ≈534k. Training uses the Adam optimiser (learning rate 10^{-3}, batch size 32) with cross-entropy loss over 20 epochs. The model reaches 100% validation accuracy by epoch 3 (initial: 39.6%, epoch 2: 99.6%), confirming rapid feature learning on the five visually distinct map classes; final training loss is 1.8×10^{-4}.

C. Adaptive Parameter Selection

At deployment, the CNN classifies a single pre-mission map and returns scenario label $\hat{c} \in \{0, \ldots, 4\}$. Each label maps to a hand-tuned parameter set $\boldsymbol{\pi}_{\hat{c}} = \{\lambda_x, \lambda_y, \lambda_\theta, k_v, k_\omega, \phi, \alpha_\omega\}$ (Table I). A first-order exponential filter prevents abrupt gain jumps:

$$\boldsymbol{\pi}_k = (1 - \beta)\, \boldsymbol{\pi}_{k-1} + \beta\, \boldsymbol{\pi}_{\hat{c}}, \quad \beta \in (0, 1]. \tag{7}$$

The design rationale for each preset is as follows. *Noise*: widened boundary layer (ϕ=0.58) and higher smoothing (α_ω=0.965) to suppress chattering from corrupted measurements. *Disturbance*: increased λ_y and k_ω to restore heading quickly after the impulsive push. *Slip*: elevated sliding-surface gains (λ_x=λ_y=2.20) to compensate the velocity deficit. *Combined*: balances noise-smoothing and disturbance-correction requirements.

D. Stability Analysis

Consider the composite Lyapunov candidate

$$V = \tfrac{1}{2}\left(s_x^2 + s_y^2 + s_\theta^2\right) \geq 0. \tag{8}$$

Differentiating along closed-loop trajectories gives $\dot{V} = s_x\dot{s}_x + s_y\dot{s}_y + s_\theta\dot{s}_\theta$. The control law is designed so that each switching term satisfies $s_i \cdot \tanh(s_i/\phi) > 0$ for all $s_i \neq 0$, since $x\tanh(x/\phi) > 0$ for all $x \neq 0$. Substituting

TABLE I
SCENARIO-SPECIFIC SMC PARAMETER SETS

Param.	Norm.	Noise	Dist.	Slip	Comb.
λ_x	2.00	2.00	2.15	2.20	2.15
λ_y	2.00	2.10	2.30	2.20	2.35
λ_θ	1.00	0.95	1.05	1.00	1.00
k_v	0.30	0.30	0.33	0.35	0.35
k_ω	0.80	0.78	0.88	0.86	0.88
ϕ	0.50	0.58	0.52	0.62	0.62
α_ω	0.950	0.965	0.945	0.960	0.965

(4) and (3) into the closed-loop error dynamics and grouping bounded disturbances and modelling residuals into $D(t)$, with $\|D(t)\| \le \bar{d}$, yields

$$\dot{V} \le -k_\omega \sum_{i\in\{y,\theta\}} s_i \tanh\left(\frac{s_i}{\phi}\right) + \bar{d}\,\|s\|. \tag{9}$$

Because $|\tanh(\cdot)| \le 1$ and $x\tanh(x/\phi) \ge |x|\tanh(1) \approx 0.762|x|$ for $|x| \ge \phi$, we have $\dot{V} < 0$ whenever $\|s\| > \bar{d}/(0.762\,k_\omega)$, establishing *uniform ultimate boundedness* (UUB) of the sliding surfaces. Inside the boundary layer $|s_i| \le \phi$, the tanh acts as a smooth gain that bounds the steady-state tracking error by

$$\|e_i\|_\infty \le \frac{\phi + \bar{d}/(\lambda_i k_\omega)}{\lambda_i}. \tag{10}$$

Equation (10) reveals the design trade-off exploited by the adaptive parameter selection: a wider ϕ reduces chattering (smaller $\dot{V}$ oscillations) at the cost of a larger ultimate error bound, which motivates the scenario-specific ϕ values in Table I.

V. SIMULATION EXPERIMENTS AND RESULTS

A. Simulation Setup

Implementation. All experiments are implemented in Python 3 using NumPy for kinematic simulation, PyTorch for CNN training and inference, and Matplotlib for visualisation. Simulations are fully reproducible via fixed random seeds and centralised YAML configuration.

Both controllers track a straight-line reference at $v_r = 0.3\,\text{m/s}$ for $T = 20\,\text{s}$ ($\Delta t = 0.01\,\text{s}$, 2000 steps). The robot starts at the origin $(x_0, y_0, \theta_0) = (0, 0, 0)$, aligned with the trajectory, so all initial tracking errors are zero; deviations arise solely from the injected perturbations. Injected conditions:

- **Normal:** no perturbation.
- **Noise:** zero-mean Gaussian noise on (x, y, θ) measurements ($\sigma_{xy} = 0.02\,\text{m}$, $\sigma_\theta = 0.01\,\text{rad}$).
- **Disturbance:** single external push of $+0.4\,\text{m}$ in y at $t = 8\,\text{s}$.
- **Slip:** 30% velocity reduction during $t \in [10, 14]\,\text{s}$.
- **Combined:** noise, disturbance, and slip simultaneously.

Metrics: RMSE tracking error, final tracking error at $t = T$, total control effort ($\sum |\omega_c|$), and chattering index (variance of successive ω_c differences).

TABLE II
CLASSIFIER ACCURACY: SIMPLE VS. REALISTIC MAPS (%)

Classifier	Simple	Realistic
k-NN (k=5)	48.44	41.33
Logistic regression	99.11	72.89
Random forest	100.00	98.22
EnvironmentCNN	**100.00**	**95.11**

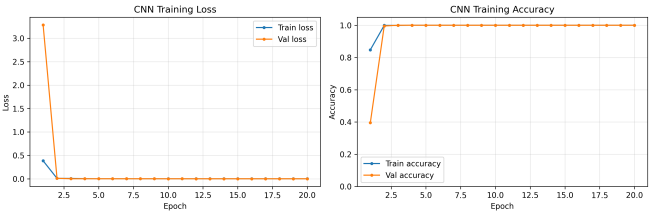


Fig. 2. CNN training and validation curves on simple maps. Loss and accuracy converge jointly by epoch 3 with no train–validation divergence, ruling out overfitting.

B. CNN Classification

The *EnvironmentCNN* achieves **100% test accuracy** on 225 held-out simple-map samples (precision/recall/F1 all 1.00; Fig. 3). Perfect scores reflect engineered class separability, not overfitting: as Fig. 2 shows, validation loss falls jointly with training loss to near-zero by epoch 3 with no divergence through epoch 20.

Same-dataset baseline comparison. Three classifiers trained on the identical split (Table II) show that perfect simple-map accuracy reflects engineered separability: random forest reaches 100%, while k-NN reaches only 48.4%.

Realistic-map evaluation. To probe robustness beyond the engineered benchmark, a second dataset of procedurally generated occupancy grids is created: each map contains border walls, three to five random rectangular obstacles, and a traversable corridor, with the class-specific cues embedded amid this clutter (speckle plus Gaussian intensity corruption for *Noise*, a high-intensity impact blob for *Disturbance*, a textured low-reflectance band for *Slip*, and all cues for *Combined*). This dataset also contains 1,500 samples (300 per class) with the same stratified 70%/15%/15% split. The CNN is retrained from scratch on the realistic training set with identical architecture and hyperparameters (not evaluated zero-shot), reaching 94.2% best validation accuracy and **95.1%** accuracy on the 225 held-out test maps (45 per class), comparable to published robot perception classifiers [11], [12].

C. Trajectory and Tracking Error

Fig. 4 gives an overview of the robot trajectories across all five scenarios for both controllers. Fig. 6 zooms into the disturbance case: the adaptive controller recovers more tightly to the reference after the $t = 8\,\text{s}$ push, reducing final error from 20.9 mm to 17.9 mm (-14.4%). Under sensor noise (Fig. 7), the angular velocity command is markedly smoother for the adaptive controller, confirming the chattering reduction

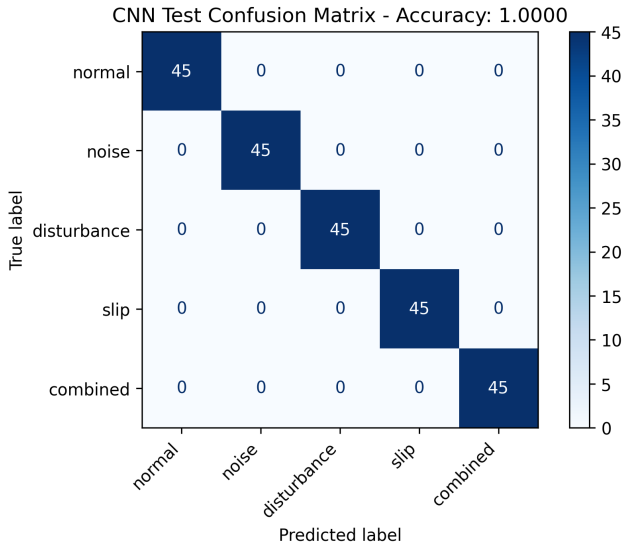


Fig. 3. CNN test-set confusion matrix on simple maps (100% accuracy, 225 samples).

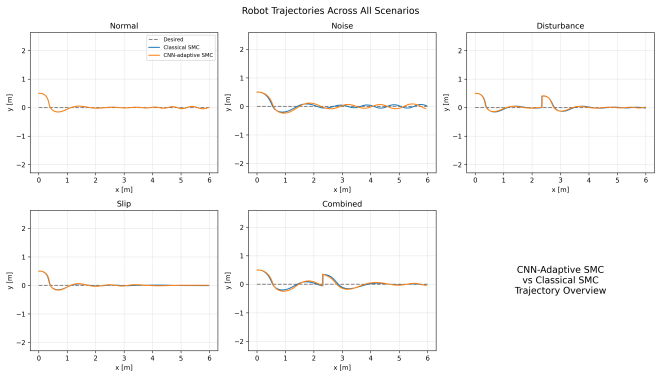


Fig. 4. Robot trajectories across all five scenarios (Classical SMC vs. CNN-Adaptive SMC). Each panel shows the reference path (black dashed) and both controller outputs. Deviations from the reference are caused solely by the injected perturbations.

predicted by the wider boundary layer. Fig. 5 shows the tracking error time series for the disturbance scenario; the faster decay after the impulse is clearly visible for the adaptive controller.

D. Quantitative Comparison

Table III reports absolute metric values for all scenarios. Table IV summarises percentage improvements of the CNN-adaptive controller over the classical baseline (positive = adaptive better). Table V compares CNN-adaptive SMC against three additional adaptive baselines evaluated under identical simulation conditions: fuzzy gain scheduling, oracle scenario selection (upper bound with perfect classification), and an RL-based preset selector. The RL baseline is a PPO agent (MLP policy, learning rate 3×10^{-4}, γ=0.99) whose observation is a five-dimensional vector (instantaneous tracking error, error rate, smoothed angular-velocity jitter, and noise/slip indicator

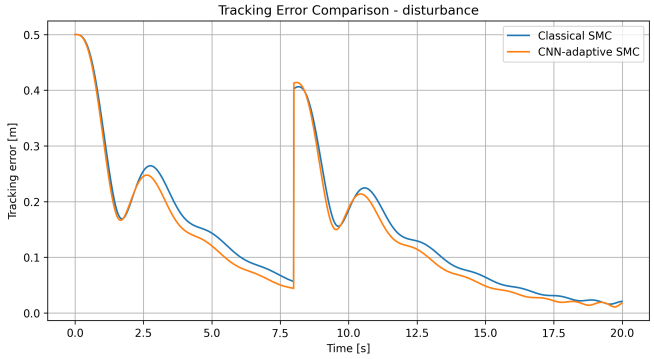


Fig. 5. Tracking error over time under external disturbance. The adaptive controller (orange) shows faster decay after the t=8 s impulse compared with the classical controller (blue).

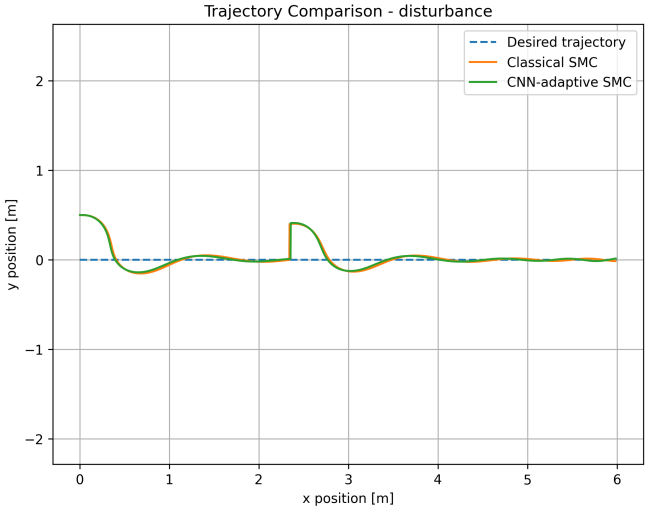


Fig. 6. Trajectory comparison under external disturbance. The CNN-adaptive controller (orange) recovers more tightly to the reference after the impulse at $t = 8$ s.

flags), whose discrete action selects one of the five parameter presets of Table I every 0.2 s, and whose reward is $-e - 0.05\, j_\omega - 0.001\,|\omega_c|$, penalising tracking error e and chattering proxy j_ω. Training uses 80k environment steps (40 episodes with randomised scenarios), after which the episode return had plateaued. Fig. 8 and Fig. 9 visualise these results.

E. Discussion

Alternative adaptive controllers. Table V benchmarks fuzzy gain scheduling [5], oracle scenario selection (perfect map classification), and a PPO policy trained for 80k steps to pick SMC presets. Fuzzy-SMC reduces chattering modestly under noise (70.5 vs. 76.5) but underperforms CNN-adaptive and oracle on disturbance and slip recovery. Oracle selection establishes the ceiling: 15.6 mm final error under disturbance vs. 17.9 mm for CNN-adaptive and 27.4 mm vs. 27.4 mm under slip, confirming that learned map classification is near-optimal on this benchmark. RL-based selection achieves the lowest combined-scenario chattering (48.9) but degrades under

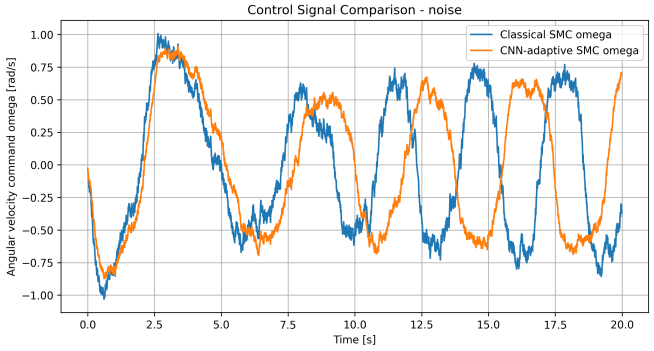


Fig. 7. Angular velocity command ω_c under sensor noise. The CNN-adaptive controller (orange) produces significantly smoother commands than the classical controller (blue).

TABLE III
ABSOLUTE CONTROLLER METRICS PER SCENARIO

Scenario	Ctrl.	RMSE (m)	Final (m)	Effort	Chatter
Normal	Classical	0.1465	0.0244	803.05	19.24
	Adaptive	0.1465	0.0244	803.05	19.24
Noise	Classical	0.1599	0.0437	775.83	76.48
	Adaptive	0.1786	0.1008	730.51	49.14
Disturbance	Classical	0.1876	0.0209	874.45	15.74
	Adaptive	0.1780	0.0179	923.15	17.83
Slip	Classical	0.1796	0.0407	577.12	7.88
	Adaptive	0.1743	0.0274	640.25	9.72
Combined	Classical	0.2357	0.0650	673.99	70.87
	Adaptive	0.2378	0.0642	705.86	51.65

disturbance and slip, indicating that more training data and reward shaping are needed before RL can replace the CNN supervisor.

Normal. Both controllers use identical parameters (Normal preset in Table I), so performance is equivalent by construction. This serves as a sanity check: the CNN correctly classifies the unperturbed map and the parameter filter introduces no steady-state offset.

Noise. Adaptive SMC reduces the chattering index by **35.7%** (from 76.48 to 49.14) via the wider boundary layer (ϕ=0.58) and stronger angular-velocity smoothing (α_ω=0.965). As predicted by (10), increasing ϕ widens the UUB error ball: RMSE rises by 11.7% and the final steady-state error grows from 43.7 mm to 100.8 mm because smooth actuation is prioritised over tight position holding under persistent measurement noise. This trade-off is intentional for actuator longevity and is consistent with the oracle preset, which exhibits the same pattern (49.3 chattering index, 103.5 mm final error).

Disturbance. Stronger correction gains (k_ω=0.88, λ_y=2.30) enable the adaptive controller to recover after the impulse, improving final tracking error by **14.4%** (20.9 mm $\rightarrow$ 17.9 mm). RMSE also improves by 5.1%; control effort increases by 5.6% due to the more aggressive gains. The

TABLE IV
CNN-ADAPTIVE IMPROVEMENT OVER CLASSICAL SMC (%). †UNDER NOISE THE FINAL ERROR INCREASES IN ABSOLUTE TERMS FROM 43.7 MM TO 100.8 MM; THIS IS THE INTENTIONAL SMOOTHNESS–ACCURACY TRADE-OFF OF (10), NOT A REGRESSION.

Scenario	RMSE	Final Err.	Effort	Chattering
Normal	0.00	0.00	0.00	0.00
Noise	−11.65	−130.62†	+5.84	**+35.75**
Disturbance	+5.12	**+14.42**	−5.57	−13.32
Slip	+2.94	**+32.69**	−10.94	−23.40
Combined	−0.85	+1.20	−4.73	**+27.12**

TABLE V
MULTI-CONTROLLER COMPARISON (SELECTED METRICS)

Scenario	Metric	Class.	Fuzzy	CNN	Oracle	RL
Noise	Chattering index	76.48	70.50	**49.14**	49.32	50.72
Disturbance	Final err. (mm)	20.92	24.75	17.90	**15.58**	32.75
Slip	Final err. (mm)	40.73	30.39	**27.41**	27.44	41.60
Combined	Chattering index	72.79	71.66	51.65	51.37	**48.88**

adaptive controller settles within the 20 mm threshold at t=18.3 s, whereas the classical controller does not fully settle within the 20 s window.

Slip. Elevated sliding-surface gains compensate the 30% velocity deficit, yielding a **32.7%** improvement in final error (40.7 mm $\rightarrow$ 27.4 mm). Control effort rises by 10.9% due to the higher gains, and chattering worsens by 23.4%, which is a known consequence of increasing sliding-surface aggressiveness, consistent with the UUB bound in (10). The gain increase is a deliberate design choice: under slip, precise position recovery is prioritised over smoothness.

Combined. The hardest scenario: adaptive reduces chattering by **27.1%** and marginally improves final error (+1.2%), while RMSE degrades by 0.9%. The Combined preset (Table I) compromises between noise-smoothing and disturbance-correction requirements. This confirms that a single static parameter set cannot simultaneously resolve all concurrent effects; continuous on-line adaptation remains future work.

Computational considerations. The proposed scheme adds no runtime burden to the control loop: the CNN performs a single pre-mission forward pass of a ≈534k-parameter network, after which the controller executes the same closed-form update (3)–(5) as classical SMC at every 10 ms step. By contrast, the fuzzy baseline evaluates rule memberships at every control step and the RL baseline requires a policy forward pass every 0.2 s, so the CNN-supervised architecture is also the cheapest of the adaptive variants at deployment time. All five controllers run faster than real time on a laptop-class CPU in our simulation.

VI. CONCLUSION

This paper presented a CNN-adaptive SMC framework for trajectory tracking of a simulated differential-drive robot under five uncertainty conditions. A lightweight *EnvironmentCNN* (≈534k parameters) classifies the operating environment from

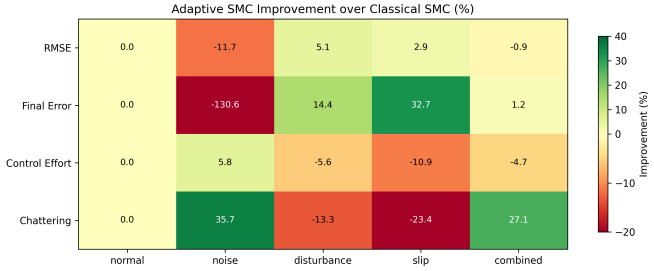


Fig. 8. Improvement heatmap of CNN-adaptive SMC over classical SMC (%). Green = adaptive better; red = adaptive worse.

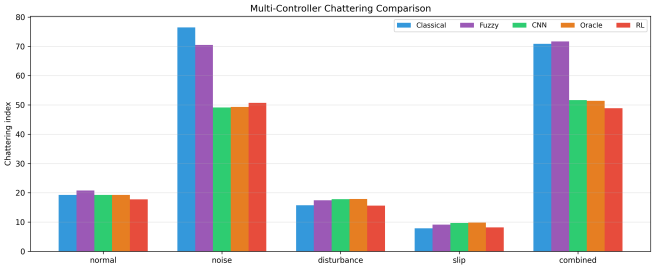


Fig. 9. Chattering index across five controllers and scenarios (lower is better). CNN-adaptive and oracle presets achieve the largest reductions under noise and combined uncertainty relative to classical SMC; RL is competitive under combined uncertainty.

a 64×64 occupancy map image and selects a corresponding SMC parameter set at run time, with exponential smoothing to prevent gain discontinuities. A Lyapunov-based UUB analysis confirms that the closed-loop system is uniformly ultimately bounded under bounded disturbances, and that the steady-state error bound is directly parameterised by the boundary-layer width ϕ, providing theoretical grounding for the scenario-specific presets.

Simulation results across five scenarios confirm:

- **35.7%** chattering reduction under sensor noise (76.48 $\rightarrow$ 49.14), with expected steady-state error widening per (10).
- **14.4%** improvement in final tracking-error recovery after external disturbance (20.9 mm $\rightarrow$ 17.9 mm), within 2.3 mm of the oracle upper bound.
- **32.7%** improvement in final error under wheel slip (40.7 mm $\rightarrow$ 27.4 mm).
- Near-oracle performance across fuzzy, RL, and classical baselines (Table V), with 95.1% CNN accuracy on realistic maps.

Limitations. The CNN operates on synthetic maps with engineered class separability (Section V) and performs a single pre-mission classification rather than continuous on-line inference. Adaptive parameters are hand-tuned per scenario rather than learned end-to-end, limiting generalisation to unseen conditions. The evaluation is restricted to a single straight-line trajectory in a 2-D kinematic simulation, without wind, uneven terrain, or non-Gaussian noise, and each scenario is evaluated with a single fixed random seed rather than averaged over Monte-Carlo repetitions.

Future work will address these limitations in five directions: (i) replace synthetic maps with real LiDAR-based occupancy grids (realistic-map CNN already reaches 95.1%); (ii) deploy continuous on-line scenario inference so that the controller can react to mid-mission condition changes; (iii) extend RL training with larger scenario randomisation and multi-objective rewards; (iv) evaluate on curved and waypoint trajectories with Monte-Carlo repetitions to attach confidence intervals to the reported improvements; (v) validate on a physical differential-drive robot.

Code Availability

The complete simulation framework, CNN training pipeline, and dataset generation scripts will be made available upon paper acceptance and at the camera ready stage.

References

[1] V. I. Utkin, *Sliding Modes in Control and Optimization*. Springer-Verlag, Berlin, 1992.

[2] C. Edwards and S. K. Spurgeon, *Sliding Mode Control: Theory and Applications*. Taylor & Francis, London, 1998.

[3] Y. Shtessel, C. Edwards, L. Fridman, and A. Levant, *Sliding Mode Control and Observation*. Birkhäuser, New York, 2014.

[4] J.-J. E. Slotine and W. Li, *Applied Nonlinear Control*. Prentice Hall, Englewood Cliffs, NJ, 1991.

[5] F.-J. Lin and P.-H. Shen, "Robust fuzzy neural network sliding-mode control for two-axis motion control system," *IEEE Trans. Ind. Electron.*, vol. 53, no. 4, pp. 1209–1225, Jun. 2006.

[6] S. S. Ge and C. Wang, "Adaptive neural control of uncertain MIMO nonlinear systems," *IEEE Trans. Neural Netw.*, vol. 15, no. 3, pp. 674–692, May 2004.

[7] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, May 2015.

[8] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *J. Mach. Learn. Res.*, vol. 17, no. 39, pp. 1–40, 2016.

[9] N. P. Nguyen and S. K. Hong, "Sliding mode Thau observer for actuator fault diagnosis of quadcopter UAVs," *Appl. Sci.*, vol. 8, no. 10, p. 1893, 2018.

[10] D. Liberzon, *Switching in Systems and Control*. Birkhäuser, Boston, MA, 2003.

[11] Y. Ran, X. Xu, M. Luo, J. Yang, and Z. Chen, "Scene classification method based on multi-scale convolutional neural network with long short-term memory and whale optimization algorithm," *Remote Sens.*, vol. 16, no. 1, p. 174, Jan. 2024.

[12] A. Demirtaş, G. Erdemir, and H. Bayram, "Indoor surface classification for mobile robots," *PeerJ Comput. Sci.*, vol. 10, p. e1730, Jan. 2024.